26-1 프로젝트 수강신청서 및 계획서 내용

> 이 문서는 `25-1 프로젝트 수강신청서 및 계획서 - 양식.docx` 에 작성할 내용입니다.

1. 기본 정보

항목	내용
학번	C235180
성명	서민지
학과	컴퓨터공학과
지도교수	(교수님 성함)
연락처	(이메일/전화번호)

2. 프로젝트 제목

OS 스케줄링 알고리즘을 활용한 LLM API 요청 최적화 스케줄러

영문: LLM API Request Optimization Scheduler using OS Scheduling Algorithms

3. 프로젝트 개요

3.1 연구 배경

ChatGPT, Claude 등 대규모 언어 모델(LLM) API의 사용이 급증하면서, 다중 사용자 환경에서 API 요청을 효율적으로 관리해야 하는 필요성이 대두되고 있습니다. 현재 대부분의 LLM 서비스는 단순한 선착순(FCFS) 방식으로 요청을 처리하여 다음과 같은 문제가 발생합니다:

- **응답 지연**: 긴 요청이 짧은 요청을 지연시키는 Convoy Effect
- **공정성 부재**: 모든 사용자가 동일하게 취급되어 VIP 사용자 차별화 불가
- **비용 비효율**: 우선순위 없이 처리하여 자원 낭비

3.2 해결 방안

운영체제 수업에서 학습한 CPU 스케줄링 알고리즘을 LLM API 요청 관리에 적용합니다:

1. **FCFS (First-Come, First-Served)**: 기본 선착순 처리 (베이스라인)
2. **Priority Scheduling**: 긴급 요청 우선 처리 + Aging으로 기아 방지
3. **MLFQ (Multi-Level Feedback Queue)**: 짧은 요청 최적화, 동적 우선순위
4. **WFQ (Weighted Fair Queuing)**: 사용자별 가중치 기반 공정한 자원 배분

3.3 OS 수업 연관성

OS 개념	본 프로젝트 적용
프로세스	LLM API 요청
CPU 시간	API 응답 시간
Ready Queue	요청 대기열
Context Switch	요청 전환
Starvation	장시간 대기 요청
Aging	대기 시간에 따른 우선순위 상승

4. 목표 및 기대효과

4.1 개발 목표

목표	상세 내용	측정 방법
알고리즘 구현	4가지 스케줄링 알고리즘 구현	단위 테스트 통과
REST API	요청 등록/조회/통계 API	curl 테스트
LLM 연동	Ollama 로컬 LLM 연동	실제 응답 확인

목표	상세 내용	측정 방법
성능 비교	알고리즘별 성능 측정	실험 데이터 분석

4.2 기대효과

1. **학술적 기여**
   - OS 스케줄링 이론의 실제 AI 시스템 적용 사례 제시
   - 4가지 알고리즘의 정량적 성능 비교 데이터 제공
2. **실용적 가치**
   - LLM API 서비스의 응답 시간 30% 이상 개선 (MLFQ 기준)
   - 멀티테넌트 환경에서 공정한 자원 배분 (WFQ)
3. **학습 성과**
   - OS 개념의 실제 적용 경험
   - Node.js/Express 백엔드 개발 역량

5. 기술 스택

5.1 개발 환경

분류	기술	버전	선정 이유
언어	JavaScript	ES2024	학습 용이성
런타임	Node.js	20 LTS	안정성
웹 프레임워크	Express.js	4.18	간결한 API
데이터베이스	SQLite	3.x	설치 불필요
테스트	Jest	29.x	표준 프레임워크
LLM	Ollama	최신	로컬 무료 실행

5.2 기술 선정 기준

- **단순성**: 학부생 1명이 한 학기 내 구현 가능
- **학습 가치**: OS 알고리즘 이해에 집중
- **실용성**: 실제 동작하는 시스템 구축

6. 개발 일정

주차	기간	주요 활동	산출물
1-2주	3/1 - 3/14	요구사항 분석, 설계	계획서, 설계 문서
3-4주	3/15 - 3/28	FCFS, Priority 구현	기본 스케줄러
5-6주	3/29 - 4/11	MLFQ, WFQ 구현	전체 스케줄러
7-8주	4/12 - 4/25	API 개발, 테스트	REST API, 테스트
9-10주	4/26 - 5/9	실험, 데이터 수집	성능 비교 데이터
11-12주	5/10 - 5/23	보고서 작성	최종 보고서
13주	5/24 - 5/31	발표 준비	PPT, 데모

7. 시스템 구조

7.1 전체 아키텍처

```
클라이언트 (curl/Postman)
        |
        v
Express.js REST API
  POST /requests
  GET /requests/:id
  GET /stats
        |
        v
Scheduler Manager
  +------+------+------+------+
  | FCFS |Priority|MLFQ | WFQ |
  +------+------+------+------+
        |
        v
메모리 큐 + SQLite
        |
        v
Ollama (로컬 LLM)
```

7.2 핵심 모듈

모듈	설명	예상 코드량
BaseScheduler	스케줄러 공통 인터페이스	~30줄
FCFSScheduler	선착순 스케줄러	~60줄
PriorityScheduler	우선순위 스케줄러 + Aging	~100줄
MLFQScheduler	다단계 피드백 큐	~150줄
WFQScheduler	가중치 기반 공정 큐	~120줄
합계		**~460줄**

8. 알고리즘 설계

8.1 FCFS (First-Come, First-Served)

- **원리**: 먼저 도착한 요청을 먼저 처리
- **장점**: 구현 간단, 공정함
- **단점**: Convoy Effect (긴 요청이 짧은 요청 지연)

8.2 Priority Scheduling

- **원리**: 우선순위가 높은 요청 먼저 처리
- **우선순위**: URGENT(3) > HIGH(2) > NORMAL(1) > LOW(0)
- **Aging**: 2분 이상 대기 시 우선순위 1단계 상승 (기아 방지)

8.3 MLFQ (Multi-Level Feedback Queue)

- **원리**: 4개 큐(Q0-Q3), 요청 행동에 따라 큐 이동
- **Time Quantum**: Q0(1초) < Q1(3초) < Q2(8초) < Q3(무제한)
- **MLFQ 5규칙**:
  i. 높은 큐 우선 실행
  ii. Time Quantum 초과 시 강등
  iii. 새 요청은 Q0 시작
  iv. 주기적 Boost (5초)
  v. Aging 적용

8.4 WFQ (Weighted Fair Queuing)

- **원리**: 테넌트별 가중치에 비례하여 자원 배분
- **가중치**: Enterprise(100) > Premium(10) > Standard(10) > Free(1)
- **공정성**: Jain's Fairness Index로 측정

9. 예상 성과

9.1 성능 목표

알고리즘	대기시간 개선	비고
FCFS	기준 (베이스라인)	
Priority	30%+ 개선 (URGENT)	긴급 요청 기준
MLFQ	30%+ 개선	짧은 요청 기준
WFQ	-	공정성 0.9+ 목표

9.2 품질 목표

항목	목표
테스트 커버리지	70%+
테스트 통과율	100%
코드 라인	~800줄

10. 참고 자료

1. Silberschatz, Galvin, Gagne. "Operating System Concepts" (10th Edition)
2. Remzi H. Arpaci-Dusseau. "Operating Systems: Three Easy Pieces" (OSTEP)
3. Node.js 공식 문서: https://nodejs.org/docs
4. Express.js 공식 문서: https://expressjs.com
5. Ollama 공식 문서: https://ollama.ai/docs

11. 서약

본인은 위 계획서의 내용대로 졸업프로젝트를 성실히 수행할 것을 서약합니다.

작성일: 2026년 2월

작성자: 서민지 (서명)

이 문서의 내용을 docx 양식에 복사하여 사용하세요.